

K-Nearest Neighbors (KNN)

KNN (K-Nearest Neighbors) is a simple, supervised machine learning algorithm used for both classification and regression tasks. However, it is more widely used for classification problems.

KNN works by:

- **Storing all available data** (no model training),
- And classifying a new data point based on **majority class of its K nearest neighbors** (for classification),
- Or **average of K nearest neighbor values** (for regression).

Step-by-Step Working of KNN:

1. **Choose the number K** (the number of neighbors to consider).
2. **Calculate the distance** between the new data point and all existing data points using a distance metric (typically Euclidean distance).
3. **Sort the distances** in ascending order and identify the K closest neighbors.
4. **Classify or Predict:**
 - For **classification**: Use **majority vote** among K nearest points.
 - For **regression**: Take the **mean (average)** of values of the K nearest points.

Mathematical Formula

◆ Euclidean Distance:

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

For n-dimensions:

$$\text{Distance} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Problem Statement:

We want to classify a new point P (3 , 3) using **K = 3** based on the following training data:

Point	Coordinates (x, y)	Class
A	(1, 2)	Red

B	(2, 3)	Red
C	(3, 1)	Blue
D	(6, 5)	Blue

Step 1: Calculate Euclidean Distance

The **Euclidean distance** formula is:

Calculate distances between point P (3 , 3) and each training point:

- A(1,2): $\sqrt{((3-1)^2 + (3-2)^2)} = \sqrt{(4 + 1)} = \sqrt{5} \approx \mathbf{2.24}$
- B(2,3): $\sqrt{((3-2)^2 + (3-3)^2)} = \sqrt{(1 + 0)} = \sqrt{1} = \mathbf{1.00}$
- C(3,1): $\sqrt{((3-3)^2 + (3-1)^2)} = \sqrt{(0 + 4)} = \sqrt{4} = \mathbf{2.00}$
- D(6,5): $\sqrt{((3-6)^2 + (3-5)^2)} = \sqrt{(9 + 4)} = \sqrt{13} \approx \mathbf{3.61}$

Step 2: Sort and Select K Nearest Neighbors

Sort the distances in ascending order and pick **K = 3** nearest:

1. B (1.00) → Red
2. C (2.00) → Blue
3. A (2.24) → Red

Step 3: Majority Voting

From top 3 neighbors:

- **Red:** 2 votes (B, A)
- **Blue:** 1 vote (C)

Final **Predicted Class** for point P(3,3): **Red**

Code Explanation

```
import numpy as np
import math

# Step 1: Euclidean distance function
def EucDistance(P1, P2):
    return math.sqrt((P1['x'] - P2['x'])**2 + (P1['y'] - P2['y'])**2)

# KNN Classification function
def MarvellousKNN():
    line = "-"*50
```

```

# Step 0: Training data
data = [
    {'point': 'A', 'x': 1, 'y': 2, 'label': 'Red'},
    {'point': 'B', 'x': 2, 'y': 3, 'label': 'Red'},
    {'point': 'C', 'x': 3, 'y': 1, 'label': 'Blue'},
    {'point': 'D', 'x': 6, 'y': 5, 'label': 'Blue'} ]

print(line)
print("Training data set:")
for i in data:
    print(i)
print(line)

# New point to classify
new_point = {'x': 3, 'y': 3}

# Step 1: Calculate distances
for d in data:
    d['distance'] = EucDistance(d, new_point)

print("Calculated Distances:")
for d in data:
    print(d)
print(line)

# Step 2: Sort by distance
sorted_data = sorted(data, key=lambda item: item['distance'])

print("Sorted Data by Distance:")
for d in sorted_data:
    print(d)
print(line)

# Step 3: Select top K = 3 neighbors
K = 3
k_nearest = sorted_data[:K]

# Step 4: Majority voting
red = sum(1 for i in k_nearest if i['label'] == 'Red')
blue = sum(1 for i in k_nearest if i['label'] == 'Blue')

print(f"Red votes = {red}, Blue votes = {blue}")
if red > blue:
    print("Predicted class = Red")
else:
    print("Predicted class = Blue")

# Call the function
MarvellousKNN()

```